Formation Next.js 16

TP N° 2 : Actions Server avec Next.js

SOMMAIRE

1. Prérequis : 3
2. Objectifs 3
3. Création du projet 3
4. Arborescence du projet 3
a. L'interface /types/Customer.ts 4
b. Le service /app/customers/actions.ts 5
c. La fonction /app/about/page.tsx 7
d. La fonction /app/customers/page.tsx 7
e. La fonction /app/customers/add/page.tsx 9
f. La fonction /app/customers/edit/[id]/page.tsx 9
g. La fonction /app/page.tsx 10
h. La fonction /app/layout.tsx 11
i. La fonction /app/components/Navbar.tsx 11
5. Tests 13
Conclusion 14

1. Prérequis :
   - Node.JS
   - Microsoft Visual Studio Code (VSD).
   - L'api REST suivante : https://github.com/abbouformations/api-backend.git

Récupérer la liste des clients	GET + /customers
Récupérer un client par son ID	Get + /customers/{id}
Ajouter un nouveau client	POST + /customers
Modifier un client	PUT + /customers/{id}
Supprimer un client	DELETE + /customers/{id}

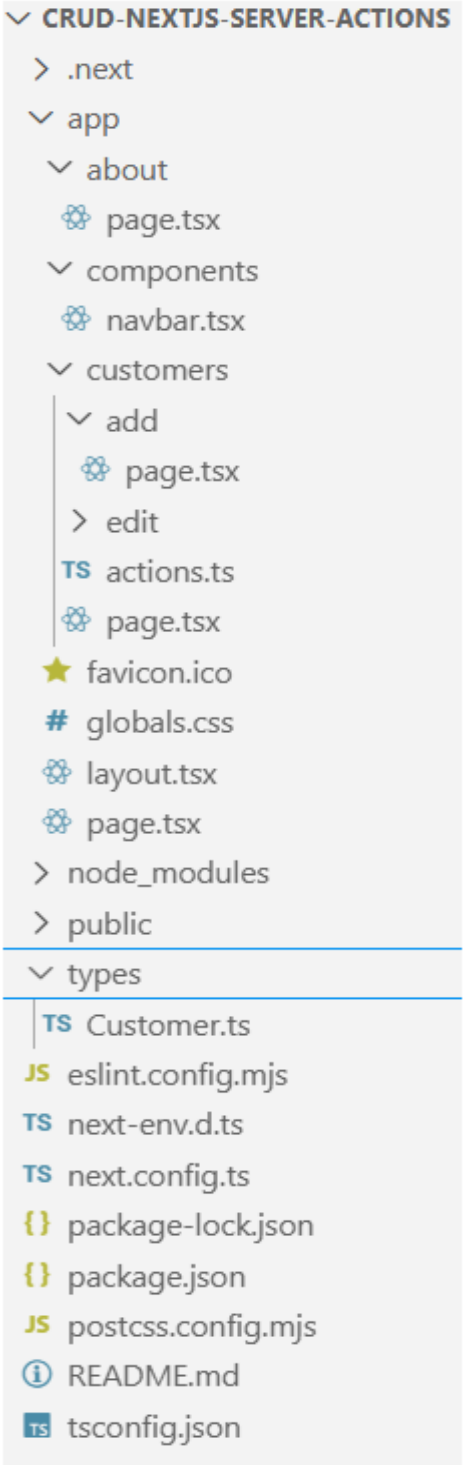
2. Objectifs
   - ✓ Implémenter les services CRUD avec Next.js version 16.
   - ✓ Utiliser App Router fournie par Next.js à partir de la version 13.
   - ✓ Utiliser les Server Actions fournies par Next.

3. Création du projet
   - Lancer VSC.
   - Ouvrir un terminal.
   - Lancer la commande suivante :

```
PS C:\workspace_2026\EMSI\next> npx create-next-app crud-nextjs-server-actions
```

4. Arborescence du projet

   Votre projet doit avoir l'arborescence suivante (App Router) :



a. L'interface /types/Customer.ts

```
export default interface Customer {
  id?: number;
  identityRef: string;
  firstname: string;
  lastname: string;
  username: string;
}
```

b. Le service /app/customers/actions.ts

```
"use server";

import { revalidatePath } from "next/cache";
import { redirect } from "next/navigation";

const API = "http://localhost:8080/customers";

export async function getCustomers() {
  const res = await fetch(API, {
    cache: "no-store",
  });

  return res.json();
}

export async function getCustomer(id: number) {
  const res = await fetch(`${API}/${id}`);
  return res.json();
}

export async function addCustomer(formData: FormData) {
  const customer = {
    identityRef: formData.get("identityRef"),
    firstname: formData.get("firstname"),
    lastname: formData.get("lastname"),
    username: formData.get("username"),
  };

  await fetch(API, {
    method: "POST",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(customer),
  });

  revalidatePath("/customers");
  redirect("/customers");
}

export async function updateCustomer(id: number, formData: FormData) {
  const customer = {
    identityRef: formData.get("identityRef"),
    firstname: formData.get("firstname"),
    lastname: formData.get("lastname"),
    username: formData.get("username"),
  };
```

```
  await fetch(`${API}/${id}`, {
    method: "PUT",
    headers: {
      "Content-Type": "application/json",
    },
    body: JSON.stringify(customer),
  });

  revalidatePath("/customers");
  redirect("/customers");
}

export async function deleteCustomer(id: number) {
  await fetch(`${API}/${id}`, {
    method: "DELETE",
  });

  revalidatePath("/customers");
  redirect("/customers");
}
```

<u>Explications</u> :

- **"use server" :**
  - Il s'agit d'une directive qui sert à indiquer que le code doit être exécuté **côté serveur uniquement** (Server Side), et jamais dans le navigateur.

- **revalidatePath**
  - Il s'agit d'une fonction utilisée avec les **Server Actions** pour **rafraîchir automatiquement une page (ou un layout)** après une modification des données.

- **redirect :**
  - C'est une fonction qui permet de **rediriger l'utilisateur vers une autre page côté serveur**.
  - Utilisée uniquement côté serveur.
  - Ne jamais mettre de code après redirect.

- **async :**
  - sert à déclarer une fonction asynchrone**.**
  - retourne toujours **une Promise.**
  - `await` permet d'attendre le résultat d'une Promise.
- **fetch :**
  - fetch est native dans Next.js.
  - disponible :
    - ✓ côté serveur (Server Components)
    - ✓ côté client
    - ✓ dans les Server Actions
  - Pas besoin de axios.
  - Supporte : cache , revalidation , SSR

- **cache: "no-store"**

- est utilisée avec ***fetch*** pour dire ne jamais mettre cette requête en cache.
- Différences avec les autres stratégies :

cache: "force-cache"	cache (par défaut)
cache: "no-store"	jamais de cache
next: { revalidate: 10 }	cache avec refresh toutes les 10s

c. La fonction /app/about/page.tsx

```
export default function About() {
  return (
    <div>
      <h1>About Page</h1>
    </div>
  );
}
```

d. La fonction /app/customers/page.tsx

```
import { deleteCustomer, getCustomers } from "./actions";
import Link from "next/link";

export default async function CustomersPage() {
  const customers = await getCustomers();
  return (
    <div>
      <h2>Customers</h2>

      <Link href="/customers/add" className="btn btn-primary mb-3">
        Add Customer
      </Link>

      <table className="table table-bordered">
        <thead>
          <tr>
            <th>ID</th>
            <th>Identity</th>
            <th>Firstname</th>
            <th>Lastname</th>
            <th>Username</th>
            <th>Actions</th>
          </tr>
        </thead>

        <tbody>
          {customers.map((c) => (
            <tr key={c.id}>
              <td>{c.id}</td>
              <td>{c.identityRef}</td>
              <td>{c.firstname}</td>
```

```
          <td>{c.lastname}</td>
          <td>{c.username}</td>

          <td>
            <Link
              href={`/customers/edit/${c.id}`}
              className="btn btn-warning btn-sm me-2"
            >
              Edit
            </Link>

            <form
              action={deleteCustomer.bind(null, c.id)}
              style={{ display: "inline" }}
            >
              <button className="btn btn-danger btn-sm">Delete</button>
            </form>
          </td>
        </tr>
      ))}
    </tbody>
  </table>
 </div>
);
}
```

Explications :

- **deleteCustomer.bind(null, c.id)}** :
  - Vue que les « servers actions » attendent toujours des *FormData* et que dans cette méthode (deleteCustomer) on envoie un paramètre supplémentaire qui est l'id, il faut passer dans le premier paramètre *null*.

- **const customers = await getCustomers() :**
  - *await* est utilisée pour appeler une fonction asynchrone.
  - Avec *await*, il est recommandé de vérifier le résultat et gérer les erreurs :

```
async function getCustomers() {
  try {
    const res = await fetch("http://localhost:8080/customers");

    if (!res.ok) {
      throw new Error("Erreur API");
    }

    return await res.json();
  } catch (error) {
    console.error(error);
    return [];
  }
}
```

e. La fonction /app/customers/add/page.tsx

```
import { addCustomer } from "../actions";

export default function AddCustomer() {
  return (
    <form action={addCustomer}>
      <h2>Add Customer</h2>

      <div className="mb-3">
        <label>Identity</label>

        <input name="identityRef" className="form-control" />
      </div>

      <div className="mb-3">
        <label>Firstname</label>

        <input name="firstname" className="form-control" />
      </div>

      <div className="mb-3">
        <label>Lastname</label>

        <input name="lastname" className="form-control" />
      </div>

      <div className="mb-3">
        <label>Username</label>

        <input name="username" className="form-control" />
      </div>

      <button className="btn btn-success">Save</button>
    </form>
  );
}
```

f. La fonction /app/customers/edit/[id]/page.tsx

```
import { getCustomer, updateCustomer } from "../../actions";

export default async function EditCustomer({ params }: any) {
  const { id } = await params;
  const customer = await getCustomer(id);

  return (
    <form action={updateCustomer.bind(null, id)}>
```

```
    <h2>Edit Customer</h2>

    <div className="mb-3">
      <label>Identity</label>

      <input
        name="identityRef"
        defaultValue={customer.identityRef}
        className="form-control"
      />
    </div>

    <div className="mb-3">
      <label>Firstname</label>

      <input
        name="firstname"
        defaultValue={customer.firstname}
        className="form-control"
      />
    </div>

    <div className="mb-3">
      <label>Lastname</label>

      <input
        name="lastname"
        defaultValue={customer.lastname}
        className="form-control"
      />
    </div>

    <div className="mb-3">
      <label>Username</label>

      <input
        name="username"
        defaultValue={customer.username}
        className="form-control"
      />
    </div>

    <button className="btn btn-success">Update</button>
  </form>
 );
}
```

g. La fonction /app/page.tsx

```
import Link from "next/link";
```

```
export default function Home() {
  return (
    <div className="text-center mt-5">
      <h1 className="display-4 fw-bold">
        Welcome to CRUD Application using "Server actions"
      </h1>

      <p className="lead mt-3">
        Application développée avec Next.js 16, Bootstrap et Actions Server (TP
        n°2)
      </p>

      <div className="mt-4">
        <Link href="/customers" className="btn btn-primary me-3">
          View Customers
        </Link>

        <Link href="/customers/add" className="btn btn-success">
          Add Customer
        </Link>
      </div>
    </div>
  );
}
```

h. La fonction /app/layout.tsx

```
import "bootstrap/dist/css/bootstrap.min.css";
import Navbar from "./components/navbar";

export default function RootLayout({
  children,
}: {
  children: React.ReactNode;
}) {
  return (
    <html>
      <body>
        <Navbar></Navbar>

        <div className="container mt-4">{children}</div>
      </body>
    </html>
  );
}
```

i. La fonction /app/components/Navbar.tsx

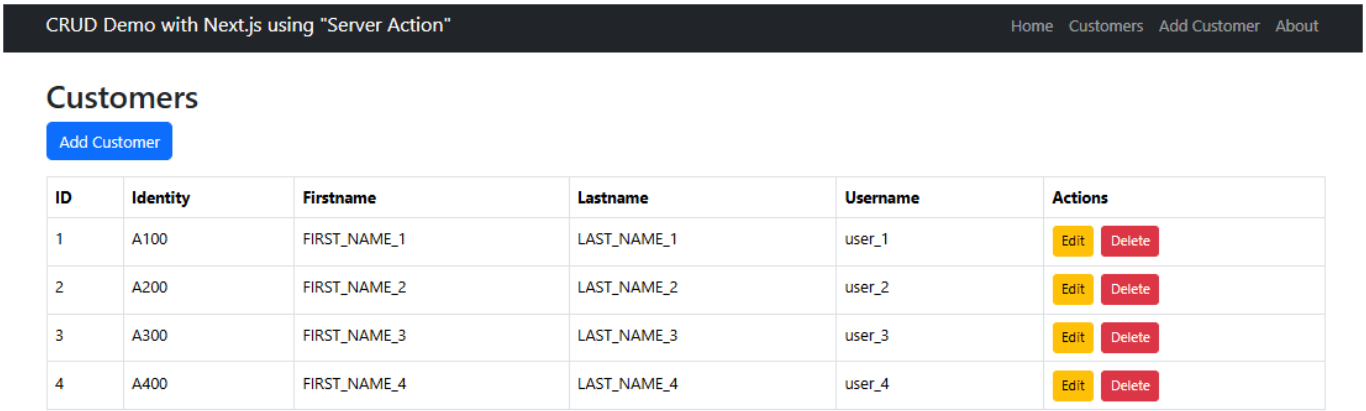
```
"use client";
```

```
import Link from "next/link";

export default function Navbar() {
  return (
    <nav className="navbar navbar-expand-lg navbar-dark bg-dark">
      <div className="container">
        <Link className="navbar-brand" href="/">
          CRUD Demo with Next.js using "Server Action"
        </Link>

        <button
          className="navbar-toggler"
          type="button"
          data-bs-toggle="collapse"
          data-bs-target="#navbarNav"
        >
          <span className="navbar-toggler-icon"></span>
        </button>

        <div className="collapse navbar-collapse" id="navbarNav">
          <ul className="navbar-nav ms-auto">
            <li className="nav-item">
              <Link className="nav-link" href="/">
                Home
              </Link>
            </li>

            <li className="nav-item">
              <Link className="nav-link" href="/customers">
                Customers
              </Link>
            </li>

            <li className="nav-item">
              <Link className="nav-link" href="/customers/add">
                Add Customer
              </Link>
            </li>

            <li className="nav-item">
              <Link className="nav-link" href="/about">
                About
              </Link>
            </li>
          </ul>
        </div>
      </div>
    </nav>
  );
}
```

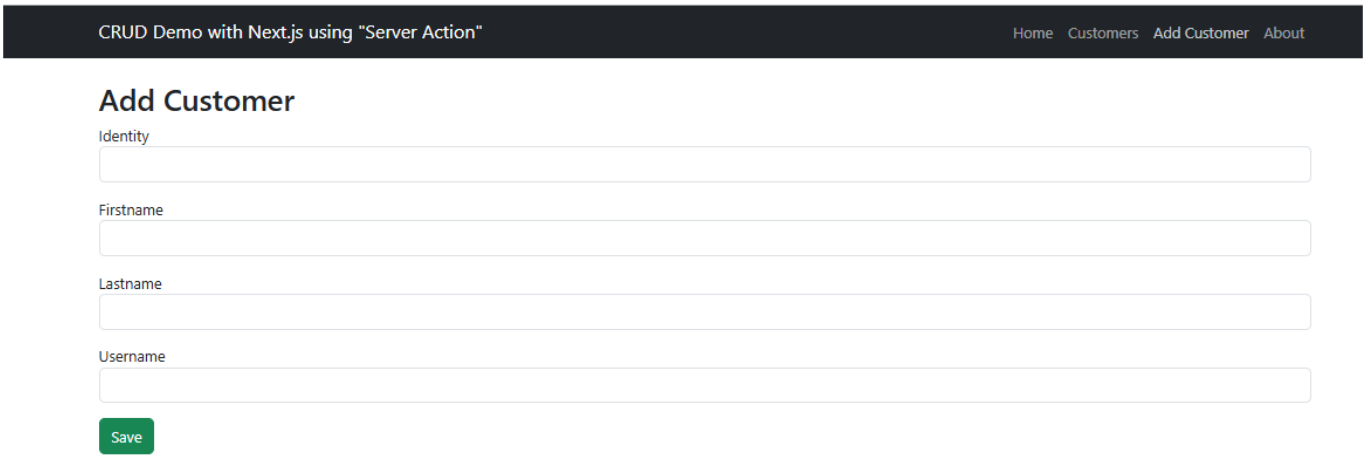
5. Tests

- Se positionner dans le dossier crud-nextjs : ***cd crud-nextjs-server-actions***
- Démarrer le serveur ***: npm run dev***
- Aller à l'URL suivante : ***http://localhost:3000***, la page suivante s'affiche :

- Cliquer sur le bouton ***« View Customers*** » pour afficher la liste des clients :

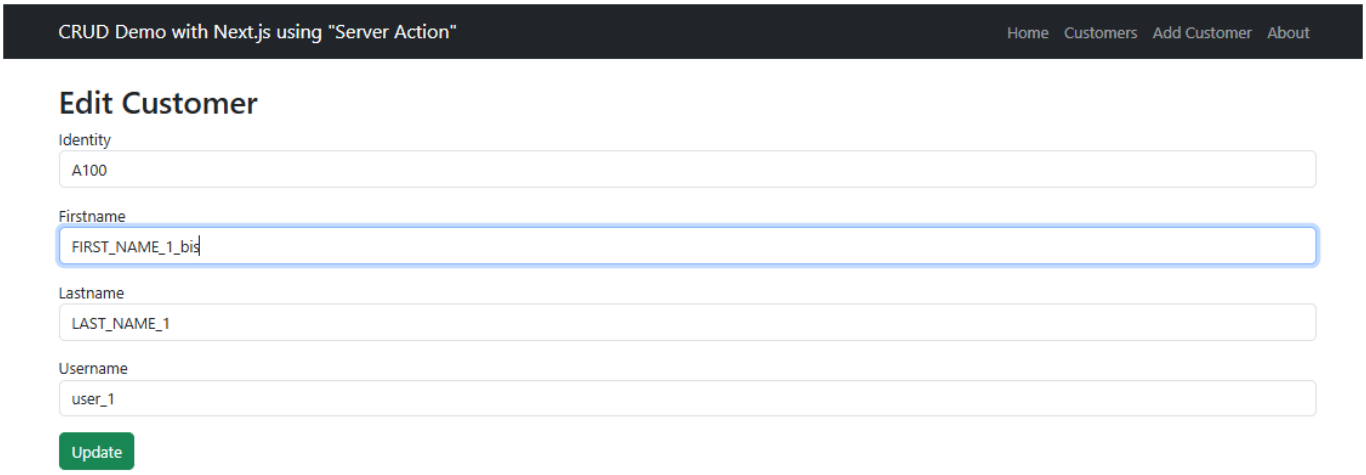


- Cliquer sur le bouton ***« Add Customers***» pour créer un nouveau client :



Entrer les informations du client et cliquer sur le bouton ***Save***.

- Cliquer sur le bouton Edit pour éditer le client afin de modifier ses informations :



Entrer les modifications souhaitées et cliquer sur le bouton ***Update***.

- Cliquer sur le bouton ***Delete*** pour supprimer le client.

Conclusion

Le code source de cet atelier est disponible sur Github :

https://github.com/abbouformations/crud-nextjs-server-actions.git